Simulador de Kernel y Gestor de Procesos en Consola

Shell & Scheduling Engine — Informe Técnico de Arquitectura (Fase I)

Integrantes: Mario Sánchez · Roberto Ferrer · Samantha Castro

Cátedra Sistemas Operativos — Prof. Marcel J. Castro G.

Universidad Católica Andrés Bello · Facultad de Ingeniería · Escuela de Ingeniería Informática — Noviembre de 2026

1. Introducción

Este informe describe la arquitectura del simulador desarrollado para la Fase I del proyecto: el núcleo ejecutor de un sistema operativo de bajo consumo, formado por una **shell interactiva de gestión** y un **motor de planificación de CPU** que se ejecutan de forma concurrente con hilos POSIX. El sistema simula la creación de procesos, su ciclo de vida, el mantenimiento del *Process Control Block* (PCB), la planificación con **Round Robin** y **SRTF apropiativo**, la carga por lotes de trazas y el cálculo de métricas de rendimiento.

El código está escrito en C++17, se compila con `g++ -pthread` (o `clang++`) mediante `Makefile` o `CMakeLists.txt`, y se ejecuta en Linux, WSL o macOS.

2. Estructura modular del código

El proyecto separa la **lógica de planificación** (determinista, sin hilos) de la **capa de concurrencia** y de la **interfaz**. Cada módulo tiene su cabecera documentada en `include/` y su implementación en `src/`.

Módulo	Responsabilidad principal	Capa
`pcb`	Estructura `PCB`, enum `ProcessState`, constantes de prioridad	Modelo
`process_table`	Dueña de los PCB (memoria dinámica con `std::unique_ptr`), PID autoincremental	Modelo
`scheduling_policy`	Interfaz `SchedulingPolicy` (patrón *Strategy*), `RoundRobinPolicy`, `SrtfPolicy`, despacho y desalojo	Planificación
`simulation`	Reloj discreto, cola de LISTOS, admisión de llegadas, `step()` de un tick, bloqueo	Planificación
`gantt` / `metrics`	Historial de ejecución por tramos; promedios de espera, retorno, respuesta y uso de CPU	Métricas
`sync`	`MutexGuard`: cerrojo RAII sobre `pthread_mutex_lock` / `pthread_mutex_unlock`	Concurrencia
`kernel`	Estado compartido: `Simulation` + `pthread_mutex_t` + `pthread_cond_t` + banderas del motor	Concurrencia
`engine`	Función del Hilo Motor (`engineThreadMain`)	Concurrencia
`cli` / `report`	Hilo CLI: tabla de comandos, validación estricta, mensajes de error; formato de `ps` y `pcb`	Interfaz
`parser` / `batch_loader`	Tokenizador, enteros estrictos, nombres; lectura y validación de trazas	Utilidades

Módulo	Responsabilidad principal	Capa
`gui` / `gui_model`	Bonus: ventana en tiempo real con raylib (dibujo) y cálculos de la vista sin dependencias gráficas (probados)	Interfaz
`main.cpp`	Crea el `Kernel` y los dos hilos; atiende la ventana gráfica; cierre ordenado y liberación de memoria	Arranque

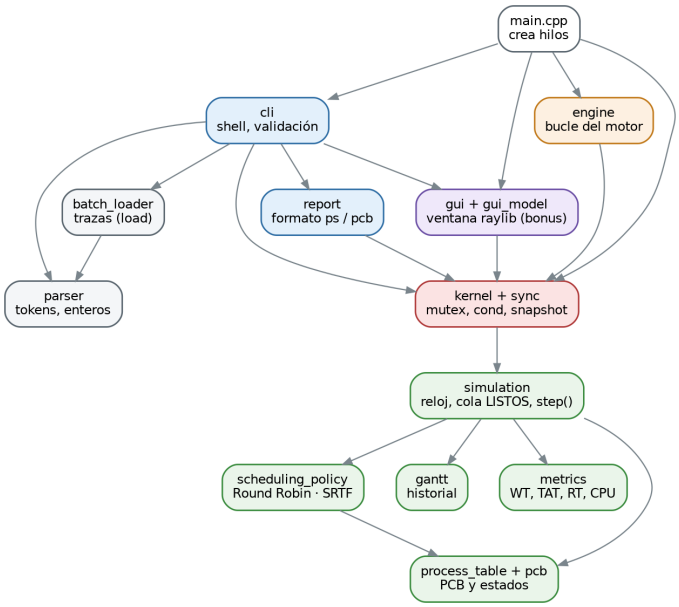


Figura 1: Dependencias entre módulos: la interfaz y el motor solo acceden al estado a través del Kernel.

La clase `Simulation` no usa hilos ni temporizadores: avanza exactamente un tick por llamada a `step()`. Esto permite verificar los algoritmos con pruebas unitarias deterministas. La clase `Kernel` envuelve a `Simulation` y agrega la sincronización; el resto del programa nunca toca el estado sin pasar por ella.

3. Arquitectura concurrente y sincronización

Hilos. El hilo principal crea el `Kernel` y lanza dos hilos con `pthread_create`: el **Hilo CLI** (`cliThreadMain`), que lee y valida comandos, y el **Hilo Motor de Scheduling** (`engineThreadMain`), que simula el reloj de CPU. Al terminar la CLI (`exit` o fin de la entrada), `main` pide el apagado del motor, lo espera con `pthread_join` y libera la memoria dinámica de los PCB.

Exclusión mutua. Un único `pthread_mutex_t` protege **todo** el estado compartido: tabla de procesos, cola de LISTOS, reloj, historial de Gantt y banderas de control. Cada método público del `Kernel` abre una sección crítica con `MutexGuard`, que llama a `pthread_mutex_lock` en su constructor y a `pthread_mutex_unlock` en su destructor, de modo que el cerrojo se libera en todos los caminos de salida. Al existir un solo cerrojo, que nunca se toma de forma anidada, no puede haber espera circular y, por tanto, tampoco interbloqueos.

Variable de condición. Cuando el reloj está en pausa o no quedan procesos sin terminar, el motor espera en `pthread_cond_wait`, que libera el mutex mientras duerme. La espera está dentro de un `while` que vuelve a comprobar la condición, lo que cubre los despertares espurios. La CLI despierta al motor con `pthread_cond_signal` en `start`, `create`, `load` y `unblock`, y con `pthread_cond_broadcast` en `exit`.

Retardo real fuera de la sección crítica. Cada tick se ejecuta completo con el mutex tomado (`runNextTick` → `step`), lo que dura microsegundos. El retardo de `set_speed` se hace después y **sin** el mutex, con `usleep` en

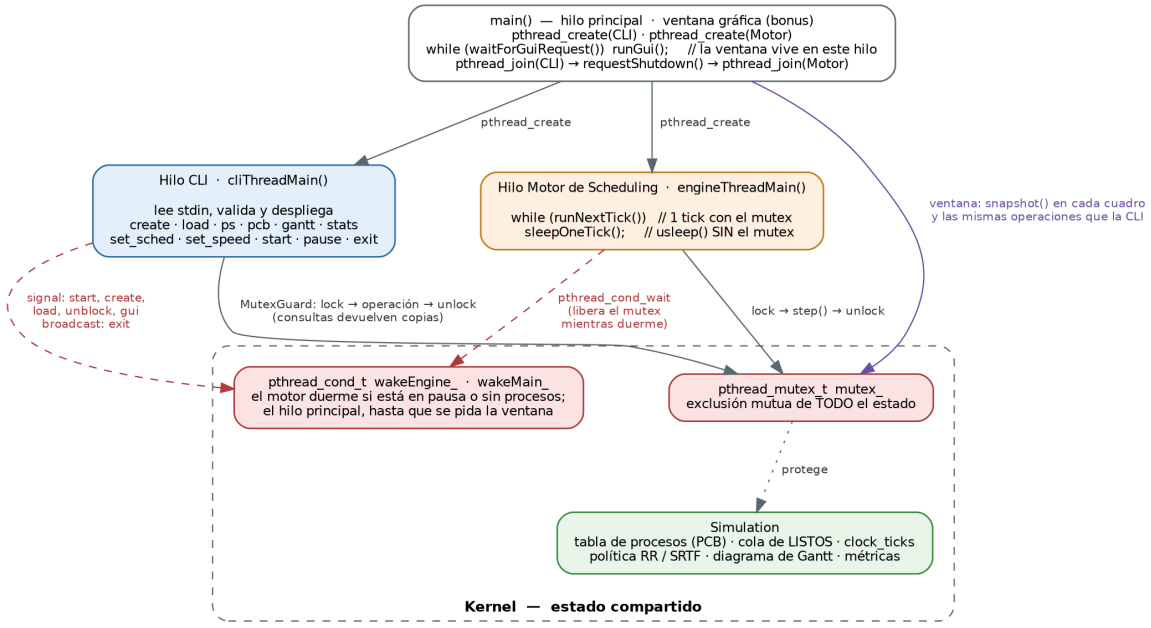


Figura 2: Hilos POSIX y mecanismos de sincronización.

tramos de 10 ms; entre tramos se consultan las banderas, de modo que `pause`, `exit` y los cambios de velocidad surten efecto en menos de 10 ms aunque el tick dure 5000 ms. La duración del tick se mide con un reloj monótono, no sumando los tramos pedidos, así que dura lo configurado aunque el sistema alargue cada `usleep`. Así la CLI nunca queda bloqueada por el motor.

Consultas por copia. `ps`, `pcb`, `gantt` y `stats` obtienen una copia consistente del estado (`SystemSnapshot`) en una sola sección crítica y le dan formato fuera de ella. Solo el Hilo CLI escribe en la consola, por lo que las salidas no se mezclan.

Ventana gráfica (bonus). El comando `gui` abre una ventana hecha con raylib que muestra en tiempo real la tabla de procesos, la CPU, las colas, las métricas y el diagrama de Gantt, y permite iniciar o pausar el reloj y cambiar la política, el quantum, el envejecimiento y la velocidad. La ventana vive en el **hilo principal**, que antes solo esperaba en `pthread_join` (en macOS es obligatorio crearla y atenderla allí). El Hilo CLI no la abre: la pide con `Kernel::requestGui()`, que hace `pthread_cond_signal` sobre una segunda variable de condición; el hilo principal dormía en `pthread_cond_wait` (`waitForGuiRequest`) y despierta para abrirla, igual que con `exit`. En cada cuadro la ventana toma una copia con `Kernel::snapshot()`, bajo el mutex, y dibuja esa copia; sus botones llaman a los mismos métodos que la CLI. No hay estado compartido nuevo fuera del `Kernel`. La interfaz es opcional (`make gui` o `cmake -DSIM_GUI=ON`): sin ella el proyecto no depende de raylib. Cómo instalar la librería está en el Anexo B.

La ausencia de condiciones de carrera se verificó con **ThreadSanitizer** (`make tsan`), que ejecuta las pruebas unitarias —incluida una prueba de estrés con consultas, creaciones y cambios de política concurrentes— y los escenarios completos sin reportar ningún acceso conflictivo.

4. Modelo de procesos (PCB)

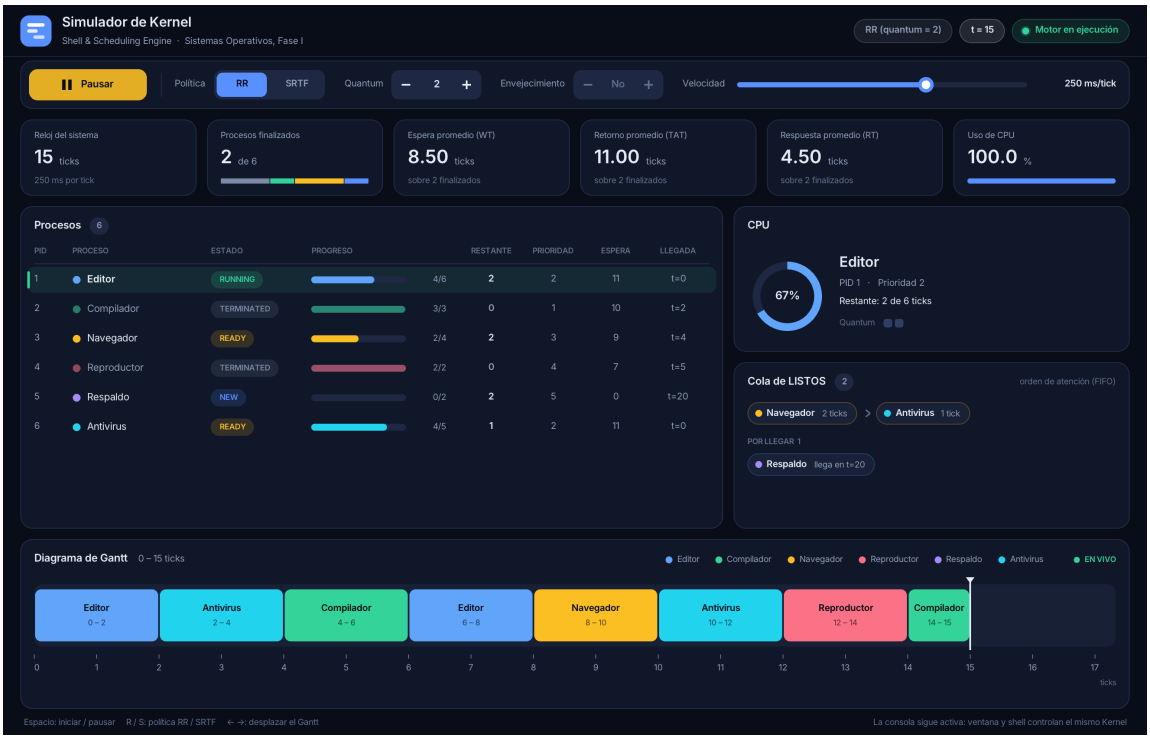


Figura 3: Interfaz gráfica del comando `gui` (bonus): Round Robin con quantum 2 a mitad de la ejecución.

Campo del PCB	Significado
`pid`, `name`, `priority`	Identidad (PID autoincremental desde 1), nombre sin espacios y prioridad 1 (más alta) a 5
`state`	NEW, READY, RUNNING, BLOCKED o TERMINATED
`arrivalTime`	Tick de llegada al sistema
`burstTotal`, `burstRemaining`	Ráfaga inicial y ráfaga restante
`startTime`, `finishTime`	Primer uso de CPU y finalización (−1 mientras no ocurren)
`waitingTime`	Ticks acumulados en la cola de LISTOS
`quantumUsed`	Ticks consumidos en la porción actual (Round Robin)

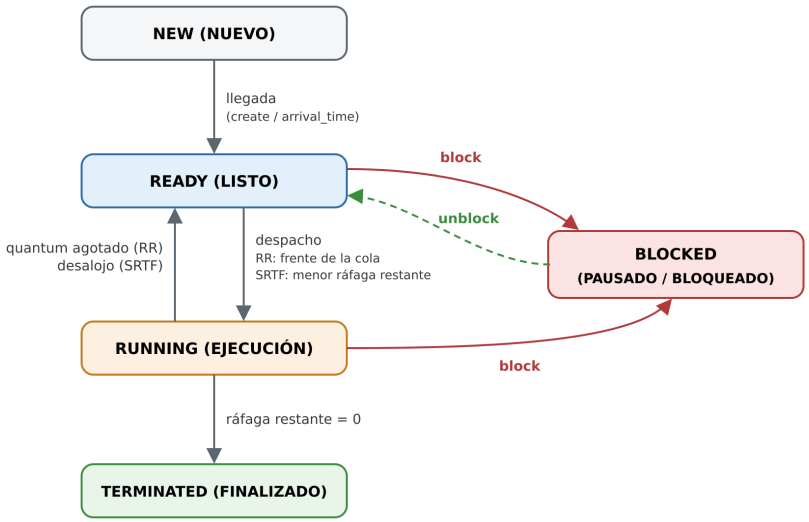


Figura 4: Ciclo de vida del proceso y eventos que producen cada transición.

Un proceso creado con `create` llega en el instante actual y pasa de NEW a READY de inmediato. Uno cargado con `load` permanece en NEW hasta que el reloj alcanza su tiempo de llegada. Los comandos `block` y `unblock` implementan el estado PAUSADO/BLOQUEADO: un proceso bloqueado sale de la CPU o de la cola y, al

continúa con un quantum nuevo. Las llegadas no desalojan al proceso en ejecución.

SRTF apropiativo. Siempre ejecuta el proceso con menor ráfaga restante. Si llega o se desbloquea un proceso con ráfaga restante estrictamente menor que la del proceso en ejecución, este es desalojado de inmediato. Los empates se resuelven por prioridad (criterio de prioridades apropiativo), luego por llegada y por PID; con igual ráfaga e igual prioridad no se desaloja, para evitar cambios de contexto inútiles.

Inanición y envejecimiento (opcional). SRTF es un caso particular de la planificación por prioridades en el que la prioridad es la ráfaga restante (Silberschatz, §5.3.3), así que un proceso largo puede postergarse indefinidamente mientras sigan llegando procesos cortos. El comando `set_aging N`, desactivado por defecto, aplica a esa clave la técnica de envejecimiento del libro: la política compara la *ráfaga efectiva* = ráfaga restante − (espera en LISTOS / N), con división entera, tanto al elegir candidato como al decidir un desalojo. Subir la prioridad (1–5) no serviría, porque en SRTF solo desempata a igual ráfaga. La espera acumulada no se pierde al obtener la CPU, de modo que el proceso envejecido no es desalojado en cadena. La ráfaga restante real no se modifica: solo cambia el orden de selección y, con él, el Gantt y las métricas. Con la traza `workloads/inanicion_srtf.txt` (un proceso de ráfaga 6 frente a ocho de ráfaga 2 que llegan cada 2 ticks), SRTF puro posterga al proceso largo 16 ticks y `set_aging 2` lo atiende en $t = 8$: la espera máxima baja de 16 a 8 ticks, a cambio de que la espera media suba de 1,78 a 3,56. Se mantiene desactivado por defecto porque con N = 1 el Escenario 2 ya no da el Gantt exigido.

Cambio en caliente. `set_sched` reemplaza el objeto de política bajo el mutex; el proceso en ejecución inicia una porción nueva y la siguiente decisión ya usa el algoritmo nuevo.

Métricas. Para cada proceso finalizado: retorno TAT = instante de fin − llegada; espera WT = ticks en la cola de LISTOS (igual a TAT − ráfaga si nunca se bloqueó); respuesta RT = primer uso de CPU − llegada. `stats` muestra sus promedios sobre los procesos finalizados y el uso de CPU = ticks ocupados / ticks simulados × 100.

Escenario 2 (SRTF, carga por lotes). En $t = 1$ llega Job_B (ráfaga 2) y desaloja a Job_A, al que le quedan 3. El Gantt obtenido es exactamente el exigido por el enunciado: `[0-1s: Job_A] -> [1-3s: Job_B] -> [3-4s: Job_C] -> [4-7s: Job_A]`.

Proceso	Llegada	Ráfaga	Prioridad	Fin	TAT	WT	RT
Job_A	0	4	2	7	7	3	0
Job_B	1	2	1	3	2	0	0
Job_C	3	1	3	4	1	0	0
Promedio					**3,33**	**1,00**	**0,00**

Escenario 1 (Round Robin, quantum 2). Sin interrupciones el Gantt es `[0-2s: P1] -> [2-4s: P2] -> [4-6s: P3] -> [6-8s: P1] -> [8-9s: P2] -> [9-10s: P1]`. Si se pausa en $t = 3$ y se cambia a SRTF, el resultado es `[0-2s: P1] -> [2-5s: P2] -> [5-7s: P3] -> [7-10s: P1]`; ambos casos están cubiertos por pruebas automáticas.

Proceso	Llegada	Ráfaga	Prioridad	Fin	TAT	WT	RT
P1	0	5	2	10	10	5	0
P2	0	3	1	9	9	6	2
P3	0	2	3	6	6	4	4
Promedio					**8,33**	**5,00**	**2,00**

6. Shell interactiva y validación de entradas

El Hilo CLI separa cada línea en palabras (admite comillas dobles para rutas con espacios), busca el comando en una tabla que define cuántos parámetros acepta y su sintaxis de uso, y valida cada valor antes de tocar el `Kernel`. Los enteros se interpretan de forma estricta: se rechazan `"abc"`, `"2.5"`, `"5x"` o valores que no caben en un `int`. Ningún error detiene la simulación; además, cada comando se ejecuta dentro de un `try/catch` como red de seguridad. `help` se genera desde la misma tabla: muestra la sintaxis de cada comando y, en los que llevan parámetros, un ejemplo listo para copiar (una prueba verifica que todos los ejemplos se ejecutan sin errores de sintaxis ni de tipo).

Categoría	Cuándo se produce	Ejemplo
`[ERROR DE SINTAXIS]`	Faltan o sobran parámetros; comillas sin cerrar	`create Proceso_A`, `ps extra`
`[ERROR DE TIPO]`	Se esperaba un entero	`pcb abc`, `create P1 cinco`
`[ERROR DE PARAMETRO]`	Valor fuera de rango, algoritmo desconocido, PID inexistente	`set_sched FIFO`, `set_speed 10`, `pcb 99`
`[ERROR DE ARCHIVO]` / `[ERROR DE FORMATO]`	La traza no existe o tiene líneas inválidas (se informan todas con su número de línea y no se carga nada)	`load no_existe.txt`
`[ERROR]`	Comando no reconocido (sugiere el correcto si solo difiere en mayúsculas)	`fork`, `PS`

`load` acepta la ruta tal como la escribe el enunciado: si `load workload_test.txt` no encuentra el archivo en el directorio actual, lo busca en `workloads/` y lo informa con una línea `[INFO]`. Las rutas explícitas y los errores de archivo no cambian.

Al copiar y pegar desde un PDF o un procesador de texto se cuelan caracteres invisibles (el propio enunciado trae U+200B al final de las líneas de `workload_test.txt`); la shell y `load` los limpian antes de validar. Ctrl+C (SIGINT) no cierra la simulación: un manejador instalado con `sigaction` y `SA_RESTART` avisa que se sale con `exit`, y la lectura de la consola continúa. Además de la red de seguridad de cada comando, el hilo motor y la ventana gráfica atrapan cualquier excepción, de modo que un fallo inesperado se informa sin terminar el proceso.

Si la entrada estándar no es una terminal, la shell repite cada comando en la salida, de modo que una sesión guionizada produce una transcripción legible. Al llegar al fin de la entrada sin `exit`, el sistema se cierra igual de forma ordenada.

7. Pruebas y verificación

Verificación	Cómo se ejecuta	Resultado
98 pruebas unitarias (PCB, parser, RR —incluidos sus casos borde de quantum—, SRTF —incluidos los ejemplos de Silberschatz, p. 145, y Stallings, fig. 8.5—, envejecimiento, Gantt, métricas, Kernel con hilos, CLI y sus ejemplos de `help`, lotes, bloqueo, vista de la interfaz gráfica y su coordinación entre hilos, entradas hostiles)	`make test`	98/98

Verificación	Cómo se ejecuta	Resultado
9 escenarios contra el ejecutable real (Escenarios 1 y 2 —también con el `load` literal del enunciado—, robustez de la CLI, bloqueo y cierre, carga mixta con periodo ocioso en RR y SRTF, inanición y envejecimiento, entradas extremas y Ctrl+C)	`make e2e`	9/9
Condiciones de carrera	`make tsan` (ThreadSanitizer, unitarias + escenarios)	Sin reportes
Fugas de memoria	Valgrind sobre el Escenario 2 completo	0 bytes en uso al salir
Portabilidad y advertencias	g++ 13 y clang++ con `-Wall -Wextra -Wpedantic -Wshadow`; Makefile y CMake; `make e2e` en Linux y en macOS (bash 3.2)	Sin advertencias
Interfaz gráfica (bonus)	`make gui-check`: abre la ventana real, guarda una captura y cierra con `exit`; en la CI, sobre Linux con una pantalla virtual	Ventana, captura y cierre correctos
Integración continua	GitHub Actions en cada pull request y en cada push a `dev` y `main`	Compila, prueba, TSan, escenarios e interfaz gráfica

8. Gestión del proyecto

El repositorio usa la rama `main` para la versión de entrega, la rama `dev` para la integración y una rama `feature/*` por issue. Cada módulo se integró a `dev` mediante un pull request que cierra su issue y pasa la integración continua; `main` recibió `dev` con el PR #46, una vez aprobada la versión, y está publicada como `v1.0-fase1`. Los issues están etiquetados por módulo y por responsable, agrupados en el hito *Fase 1*, y el tablero Kanban de GitHub Projects los organiza en las columnas **Por hacer**, **Haciendo** y **Hecho**.

Responsable	Issues
Roberto Ferrer	Estructura y compilación con CI · Arquitectura concurrente (hilos y mutex) · Motor de scheduling con Round Robin · Bloqueo de procesos · Aporte individual: casos borde de Round Robin (#29)
Mario Sánchez	PCB y tabla de procesos · SRTF apropiativo y cambio en caliente · Diagrama de Gantt y métricas · Carga por lotes · Aporte individual: inanición y envejecimiento en SRTF (#30), `make e2e` en macOS (#32), `load` como en el enunciado (#35), formato exacto de `ps` (#37), interfaz gráfica (bonus, #14)
Samantha Castro	Parser y validación estricta · Comandos de la CLI · Pruebas de los escenarios · Documentación e informe · Aporte individual: escenario de carga mixta, ejemplos en `help` y cierre de la documentación (#31)

9. Referencias bibliográficas

1. Silberschatz, A., Galvin, P. B. y Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill. Capítulos 3 a 7 (gestión de procesos: procesos, hebras, planificación de la CPU, sincronización e interbloqueos); §5.3.2–5.3.3, pp. 145–146 (SRTF, prioridades, inanición y envejecimiento).
2. Tanenbaum, A. S. (2009). *Sistemas operativos modernos* (3.ª ed.). Pearson Educación. Capítulo 2 (procesos e hilos); §2.4 (planificación), p. 154 (menor tiempo restante a continuación).

3. Stallings, W. (1997). *Sistemas operativos* (2.ª ed.). Prentice Hall. Capítulos 3 (descripción y control de procesos), 4–5 (concurrencia: exclusión mutua, sincronización, interbloqueo e inanición) y 8 (planificación de monoprocesadores; figura 8.5, p. 354; SRT, p. 360).
4. Arpaci-Dusseau, R. H. y Arpaci-Dusseau, A. C. (2018). *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books. Capítulos 7–8 (planificación) y 26–30 (concurrencia, cerrojos y variables de condición).
5. Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley.
6. Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press. Capítulos 29–33 (hilos POSIX).
7. IEEE y The Open Group (2018). *IEEE Std 1003.1-2017 (POSIX.1-2017)*: `pthread_create`, `pthread_join`, `pthread_mutex_lock`, `pthread_cond_wait`, `usleep`.

Las referencias 1 a 3 son las ediciones de la bibliografía del curso (aula virtual, módulo 7).

Anexo A. Declaración del uso de Inteligencia Artificial

Herramienta utilizada. Claude (Anthropic), modelo Claude Opus 5.5, usado desde claude.ai en modo agente, con acceso a una terminal Linux y al repositorio de GitHub del equipo.

Forma de uso. La herramienta actuó como asistente de desarrollo e implementación. A partir del enunciado (PDF adjunto) y de las instrucciones del equipo, generó el código fuente, las pruebas automatizadas, este informe y la organización del repositorio: ramas, issues, pull requests y tablero. El equipo indicó sus integrantes y el flujo de trabajo en GitHub (rama `main`, rama `dev` y ramas `feature/*` integradas mediante pull requests); la herramienta propuso la división en módulos y la asignación de issues.

Prompt principal (redactado de forma ordenada a partir del mensaje original, con el mismo contenido; acompañado del enunciado en PDF):

> Crea el repositorio en GitHub, desarrolla el proyecto descrito en el enunciado adjunto y súbelo. En GitHub Projects, arma un tablero con las columnas «Por hacer», «Haciendo» y «Completado». Crea los issues necesarios y asígnalos entre Mario Sánchez, Roberto Ferrer y Samantha Castro. Todo el código debe entrar mediante pull requests. Trabaja con tres tipos de ramas: `main`, `dev` y ramas `feature/*`, donde se desarrolla el código. Los pull requests van hacia `dev`; por ahora no se integra nada en `main`.

Validación realizada durante el desarrollo. Los resultados se contrastaron con cálculos manuales y con las salidas esperadas del enunciado mediante pruebas unitarias deterministas y escenarios ejecutados contra el binario real (66 y 5 en la primera versión; 93 y 8 al cierre, ver §7); se verificó la ausencia de condiciones de carrera con ThreadSanitizer y la ausencia de fugas con Valgrind, y se compiló sin advertencias con g++ y clang++. Cada pull request ejecuta esta batería en GitHub Actions.

Uso en el aporte individual de Mario Sánchez (issues #30, #32, #35, #37 y el bonus #14). La misma herramienta, en modo agente desde Visual Studio Code (Claude Code), con acceso al repositorio local, a GitHub, al enunciado y a la bibliografía del curso. Prompt principal (resumen): analizar el issue asignado a Mario; auditar el repositorio y GitHub frente al enunciado y a la metodología de los libros del módulo 7; e implementar el aporte con el flujo de ramas y pull requests hacia `dev`. La herramienta propuso e implementó el envejecimiento, la traza de inanición, las pruebas y la documentación, y detectó los problemas que dieron origen a #32, #35 y #37. En un segundo prompt se le pidió la interfaz gráfica del bonus: la diseñó e implementó con raylib y la validó con capturas de la ventana real, pruebas unitarias de los cálculos de la vista y una prueba automática de la ventana en macOS y en Linux. Los commits se registraron con la cuenta de Mario Sánchez; el uso de la herramienta se declara en este anexo. Validación: los dos diagramas de Gantt de la traza se calcularon a mano (tabla tick a tick en el README) y se contrastaron con el simulador; las pruebas reproducen los ejemplos de Silberschatz (p. 145) y de Stallings (figura 8.5); `make test`, `make e2e` y la integración continua (ThreadSanitizer y CMake) quedaron en verde.

Uso en el aporte individual de Roberto Ferrer (issue #29, PR #42). La misma herramienta, desde la aplicación de escritorio de Claude, con acceso a una copia local del repositorio y a GitHub. Prompt principal (resumen): preparar en la rama `feature/rr-casos-borde` las pruebas de casos borde de Round Robin —quantum 1 y quantum mayor que todas las ráfagas—, explicarlas para la defensa y dejarlas listas para revisión. La herramienta escribió las dos pruebas en `tests/test_round_robin.cpp`, comprobó en una copia aparte que fallan si se altera el quantum, registró el commit con la cuenta de Roberto Ferrer (con la línea `Co-Authored-By: Claude`) y abrió el PR #42; Roberto revisó el resultado y autorizó su fusión. En la misma sesión, la herramienta compiló y probó los PR #33 a #41 y redactó las revisiones que Roberto publicó con su cuenta después de leerlas. Validación: `make test` y `make e2e` en verde y la integración continua del PR #42. Su segundo commit, la insignia de la integración continua en el README (PR #44), también lo preparó la herramienta a su pedido, con la misma línea `Co-Authored-By`.

Uso en el aporte individual de Samantha Castro (issue #31). La misma herramienta y la misma

conversación de claude.ai del prompt principal. Prompt principal (resumen): preparar en local, sin subir nada al repositorio, todo lo pendiente del issue #31 para que Samantha lo suba con su cuenta. La herramienta calculó a mano el Gantt y las métricas de `workloads/carga_mixta.txt` con RR y SRTF, escribió el escenario `tests/e2e/escenario_5_carga_mixta.sh`, agregó a `help` un ejemplo por cada comando con parámetros —con dos pruebas: que los ejemplos aparezcan y que se ejecuten sin errores de sintaxis ni de tipo; se comprobó que fallan si un ejemplo se rompe— y redactó esta actualización final del README y del informe, incluida la declaración del aporte de Roberto. Los commits se registraron con la cuenta de Samantha Castro; el uso de la herramienta se declara en este anexo. Validación: los resultados del escenario coinciden con el cálculo a mano (tabla en el README); `make test` (93/93), `make e2e` (8/8), `make tsan` y CMake en verde.

Validación y apropiación por parte del equipo (completar antes de la entrega):

- ☐ Cada integrante revisó el código de sus issues y puede explicarlo en la defensa individual.
- ☐ Se reprodujeron manualmente los Escenarios 1 y 2 y se compararon con el cálculo a mano del Gantt y las métricas.
- ☒ Se registraron aquí los prompts adicionales usados durante la revisión, si los hubo.
- ☒ Observaciones del equipo sobre los resultados aportados por la herramienta: el código generado compiló y pasó sus pruebas desde la primera versión, pero la revisión posterior contra el enunciado y en otra plataforma encontró detalles que esas pruebas no cubrían: `ps` usaba columnas de 8/16/14 caracteres y 65 guiones en lugar de 7/15/13 y 62 (corregido en el PR #38); `load workload_test.txt`, escrito tal cual como en el Escenario 2, fallaba porque la traza está en `workloads/` (PR #36); `make e2e` fallaba en macOS con bash 3.2 (PR #33); y la documentación afirmaba que el envejecimiento no altera el Gantt ni las métricas, cuando sí los altera (lo detectó la revisión de los PR y se corrigió en el #43). Conclusión: que las pruebas pasen no basta; hay que comparar cada salida con el enunciado y probar en más de una plataforma.

Anexo B. Cómo instalar la librería de la interfaz gráfica

La interfaz gráfica usa la librería **raylib**. Para descargarla usamos **Homebrew**, un instalador de paquetes: con el comando `brew` se pueden instalar varias herramientas y librerías desde la terminal. Así lo hizo el equipo en macOS:

```
brew install raylib pkgconf      # descarga la librería raylib
make gui                         # compila el simulador con la interfaz
./simulador                      # y dentro de la shell: gui
```

Si Homebrew no está instalado, se instala una sola vez con el comando que aparece en su página, brew.sh.

En Windows Homebrew no está disponible. El proyecto corre dentro de WSL, el Linux que viene con Windows (`wsl --install` en PowerShell como administrador). Allí el instalador que cumple el papel de `brew` es `apt`, y CMake descarga raylib por su cuenta:

```
sudo apt install build-essential cmake git libgl1-mesa-dev libgl1-mesa-dri \
    libx11-dev libxrandr-dev libxinerama-dev libxcursor-dev libxi-dev
git clone https://github.com/RobxrtoF/simulador-kernel-so.git && cd simulador-kernel-so
cmake -S . -B build-gui -DSIM_GUI=ON && cmake --build build-gui
./build-gui/simulador            # y dentro de la shell: gui
```

Para que la ventana se vea en Windows hace falta Windows 11 o Windows 10 actualizado.